Memory Forensics Using the Volatility Framework

A Practical Tutorial for Digital Forensic Analysis

Mohammad Zaidan
An-Najah National University
Information Security and Ethics
Instructor: Dr. Jihad Hammamreh

Abstract

Volatile memory contains a snapshot of the runtime state of a computer system and can preserve evidence that is difficult or impossible to recover from persistent storage alone. Processes, process relationships, network artifacts, loaded modules, command activity, and other operating system structures may remain available in RAM during an investigation. This paper presents a practical tutorial on the use of the Volatility Framework for memory forensics, with particular emphasis on the Volatility 2 repository supplied for this project. The paper introduces the principles of memory acquisition and analysis, explains the plugin-oriented architecture of Volatility, and demonstrates a repeatable workflow for examining Windows memory images. Common plugins such as `pslist`, `psscan`, `pstree`, `dlllist`, `netscan`, `cmdscan`, and `malfind` are discussed, together with methods for correlating their results. The paper also addresses evidence preservation, interpretation, limitations, and the transition from Volatility 2 to Volatility 3. The examples are intentionally illustrative; no claim is made that the sample output represents an actual compromised system.
Keywords: Memory forensics; Volatility; RAM analysis; digital forensics; incident response; malware analysis; Windows memory; forensic plugins.

1. Introduction

Digital investigations increasingly require analysts to examine both persistent and volatile sources of evidence. Files and logs stored on disk can reveal what happened over a period of time, but they do not always describe the complete runtime state of a system. Random Access Memory (RAM) can contain processes, network connections, loaded libraries, operating-system structures, and other artifacts that exist only while the system is running.

Memory forensics is the discipline of acquiring a memory sample and interpreting its structures to reconstruct useful evidence. It is particularly relevant to incident response and malware analysis because some forms of malicious activity are observable primarily through runtime artifacts. Volatility is an established open-source framework for this purpose. The project describes itself as an advanced memory forensics framework and provides a large plugin ecosystem for analysis of memory images [1].

The supplied project is the original Volatility repository, commonly referred to as Volatility 2. The repository is now archived and read-only, and its documented latest 2.x release is Volatility 2.6 [4, 5]. This historical version remains useful for understanding classic memory-forensics workflows and is also widely referenced in educational material.

The objective of this paper is to provide a practical, reproducible tutorial for analyzing a memory image with Volatility 2. It focuses on the workflow rather than on a fabricated forensic incident. All example outputs in this paper are explicitly illustrative and should be replaced by real output when an actual memory image is available.

2. Memory Forensics Fundamentals

2.1 Volatile Evidence

Volatile evidence is information that can change or disappear when a system is powered down or its runtime state changes. RAM is therefore different from persistent storage. At a particular point in time, RAM may contain the execution state of applications, kernel structures, cached information, and network-related objects.

A memory image is a file containing a captured representation of such volatile memory. Common acquisition sources include dedicated memory acquisition utilities, forensic agents, virtualization platforms, and operating-system crash or hibernation mechanisms. Volatility itself does not generally perform physical-memory acquisition; its primary role is analysis of an existing memory image [3].

2.2 Why Memory Analysis Matters

Memory analysis can answer questions that disk analysis alone may not answer easily:

- Which processes were active at acquisition time?
- Which processes were related through parent–child relationships?
- Which network endpoints were associated with running processes?
- Which libraries and modules were loaded?
- Was suspicious executable code present in a process?
- Did command-history structures remain in memory?

The answers should be treated as forensic observations that require correlation. A single suspicious artifact is not necessarily proof of malicious activity.

3. Volatility Framework

3.1 Project Overview

Volatility uses a plugin-oriented model. An investigator supplies a memory image and selects a plugin that understands

a particular class of operating-system artifact. The official usage documentation describes the basic command pattern as a plugin followed by the memory image and, for Volatility 2, an appropriate profile [2].

A simplified workflow is shown in Figure 1.

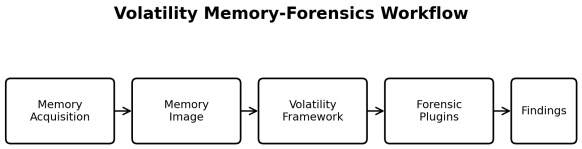


Figure 1: A practical memory-forensics workflow using Volatility.

3.2 Volatility 2 Architecture

Volatility 2 uses profiles and address-space implementations to interpret operating-system structures. Its command-line interface supports options for selecting profiles, memory images, plugins, output locations, and other analysis settings.

The official repository documentation states that Volatility 2 requires Python 2.6 or later but not Python 3.x, with some plugins requiring additional dependencies [1]. This is an important reproducibility issue: a modern Python 3 installation should not be assumed to run the original Volatility 2 code without an appropriate legacy environment.

3.3 Plugin-Oriented Model

The plugin model is one of the most useful characteristics of Volatility. Instead of manually interpreting every structure in a memory image, the analyst selects a plugin for the investigative question.

Figure 2 summarizes several classic Volatility 2 plugins.

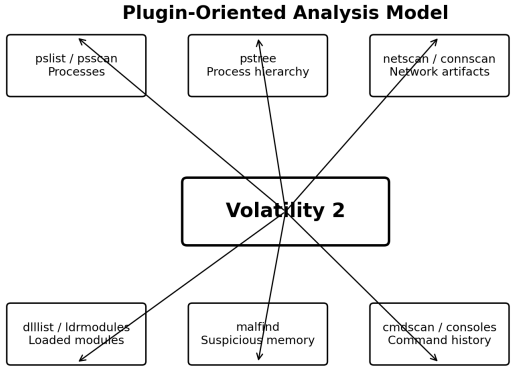


Figure 2: Examples of plugin categories in Volatility 2.

4. Installation and Basic Usage

The official repository provides a source-code installation workflow. A typical acquisition of the repository is:

```
git clone https://github.com/
volatilityfoundation/volatility.git
cd volatility
```

The project README documents the quick-start command:

```
python vol.py -h
python vol.py --info
```

The `-info` option can be used to inspect available profiles, address spaces, and plugins. The official FAQ identifies Volatility 2.6 as the latest stable 2.x release [3].

For a Windows memory image, a typical Volatility 2 command follows this pattern:

```
python vol.py pslist -f memory.raw \
  --profile=Win7SP1x64
```

The exact profile must match the memory image. Selecting an incorrect profile can produce errors or misleading results.

5. Core Forensic Plugins

5.1 `pslist`: Process Enumeration

The `pslist` plugin enumerates processes using operating-system process structures. It is often one of the first plugins executed in a Windows investigation.

```
python vol.py pslist -f memory.raw \
  --profile=Win7SP1x64
```

The output can contain process names, process identifiers, parent process identifiers, and timing information. Investigators should record the command and preserve the output as part of the case notes.

5.2 `psscan`: Process Scanning

The `psscan` plugin uses scanning techniques to locate process objects in memory. Comparing `psscan` with `pslist` can provide a broader view of process artifacts.

```
python vol.py psscan -f memory.raw \
  --profile=Win7SP1x64
```

Differences between enumeration and scanning results can be an investigative lead, but they must be interpreted according to the operating system and memory state.

5.3 `pstree`: Process Relationships

The process tree presents parent–child relationships:

```
python vol.py pstree -f memory.raw \
  --profile=Win7SP1x64
```

Process ancestry can help analysts understand how an application was started. An unexpected relationship should lead to additional evidence collection rather than an immediate conclusion.

5.4 `dlllist`: Loaded Libraries

The `dlllist` plugin lists DLLs associated with processes. It can be useful when investigating unusual applications or attempting to understand what code components were loaded into a process.

```
python vol.py dlllist -p 3108 \
  -f memory.raw --profile=Win7SP1x64
```

5.5 `netscan`: Network Artifacts

Network artifacts can connect a process to communications observed in memory. On supported Windows versions, `netscan` can be used to inspect network objects:

```
python vol.py netscan -f memory.raw \
  --profile=Win7SP1x64
```

Network evidence is especially valuable when correlated with process information and external logs.

5.6 `cmdscan`: Command History

The `cmdscan` plugin can recover certain command-history structures from memory:

```
python vol.py cmdscan -f memory.raw \
  --profile=Win7SP1x64
```

Recovered command information may provide context about user or attacker activity. Its availability depends on the memory image and operating-system state.

5.7 `malfind`: Suspicious Memory Regions

The `malfind` plugin is designed to identify potentially suspicious memory regions and characteristics associated with injected or executable code. It should be regarded as a detection aid rather than an automatic malware classifier.

```
python vol.py malfind -f memory.raw \
  --profile=Win7SP1x64
```

6. Practical Investigation

A repeatable investigation can be organized into five stages.

6.1 Stage 1: Evidence Preservation

The original memory image should be preserved as evidence. Analysts should document acquisition time, source, file size, hash values, and analysis environment where required by the investigation.

6.2 Stage 2: Profile Identification

Volatility 2 requires an appropriate profile for many analyses. The analyst can inspect available profiles using:

```
python vol.py --info
```

Profile selection should be based on reliable information about the captured system and, when possible, verified through additional memory analysis.

6.3 Stage 3: Baseline Process Analysis

The analyst can begin with:

```
python vol.py pslist -f memory.raw \
  --profile=Win7SP1x64

python vol.py psscan -f memory.raw \
  --profile=Win7SP1x64

python vol.py pstree -f memory.raw \
  --profile=Win7SP1x64
```

The goal is to establish a baseline of process activity.

6.4 Stage 4: Evidence Correlation

The most useful results often emerge by combining multiple plugins. Figure 3 illustrates this principle.

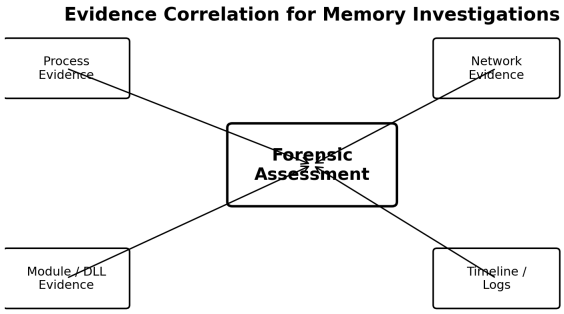


Figure 3: Correlating independent memory artifacts.

For example, a suspicious process can be examined together with its parent process, loaded DLLs, network connections, and command history. This approach is stronger than relying on one plugin output.

6.5 Stage 5: Documentation

A forensic report should record:

1. Evidence identifier and acquisition details.
2. Hash of the original memory image.
3. Volatility version.
4. Operating-system profile.
5. Commands executed.
6. Relevant output.
7. Screenshots where useful.
8. Interpretation and limitations.

7. Illustrative Analysis

The following output is intentionally illustrative. It demonstrates how an analyst could document process enumeration without claiming that the listed processes came from a real memory image.

```
$ python vol.py pslist -f memory.raw --profile=Win7SP1x64
Volatility Foundation Volatility Framework 2.6

Offset(V)        Name                 PID   PPID
---------------- -------------------- ----- -----
0x85a1c9b0       System                 4      0
0x86b3e5c0       smss.exe             268      4
0x86c7a2d0       csrss.exe            348    332
0x86d1f040       explorer.exe        2140   1992
0x86e0b7a0       example.exe         3108   2140

Illustrative output — not extracted from a real memory image.
```

Figure 4: Illustrative `pslist` output. The values are examples, not evidence from an actual memory image.

For example, if `example.exe` appeared as a child of `explorer.exe`, the observation alone would not establish that the program is malicious. The analyst should examine its executable path, parent process, loaded modules, network connections, and other available evidence.

8. Evidence Interpretation

Memory-forensic results should be separated into observations and interpretations. An observation might be: "PID 3108 is associated with `example.exe`." An interpretation might be: "The process requires further investigation because its execution context is unusual for the host."

This distinction is important in information security and digital forensics because forensic conclusions should be traceable to evidence. False positives can occur when legitimate software has unusual names, parent processes, network connections, or memory characteristics.

A useful evidence-correlation model combines process, network, module, and timeline information. The more independent artifacts support an explanation, the stronger the investigative basis becomes.

9. Security Applications

9.1 Incident Response

During incident response, memory analysis can provide a snapshot of active system state. Analysts can use Volatility to investigate processes, network artifacts, and other structures associated with a suspected compromise.

9.2 Malware Analysis

Malware can manipulate memory, inject code, create processes, or establish network connections. Volatility plugins can expose artifacts that support deeper malware analysis.

9.3 Digital Forensics

Memory evidence can complement disk and log evidence. A forensic analyst can compare memory artifacts with file-system timestamps, event logs, network records, and endpoint telemetry.

10. Advantages

Volatility has several practical advantages:

- It is open source and widely studied.
- Its plugin architecture supports many investigative questions.
- It can recover runtime artifacts unavailable from disk alone.
- It supports repeatable command-line workflows.
- It is useful for education, incident response, and malware analysis.

11. Limitations and Ethical Considerations

Memory analysis has technical and ethical limitations. The evidence is time-dependent, and acquisition quality directly affects the results. Some artifacts may be incomplete, overwritten, or unavailable.

Volatility 2 also presents an environment challenge because it was designed for Python 2 rather than modern Python 3 environments [1]. The original repository was archived on May 16, 2025, and is now read-only [4]. The latest documented 2.x release is 2.6 from December 2016 [5].

From an ethical perspective, analysts should only acquire or analyze memory when they have appropriate authorization. Memory images may contain credentials, personal information, private communications, and other sensitive data. Investigators should minimize unnecessary exposure, protect evidence, and follow applicable institutional and legal requirements.

12. Volatility 2 and Volatility 3

Volatility 3 is the modern rewrite of the project. The transition is important because commands, architecture, and installation requirements differ between the two generations. This paper intentionally uses Volatility 2 commands because the supplied project is the Volatility 2 repository.

For a modern investigation, analysts should consult the current Volatility 3 documentation rather than assume that Volatility 2 commands are interchangeable. The distinction should also be stated clearly in academic work so that another researcher can reproduce the environment.

13. Discussion

Volatility demonstrates how memory forensics converts low-level volatile data into higher-level investigative artifacts. The plugin architecture is particularly valuable because different plugins answer different questions while allowing their results to be correlated.

At the same time, a forensic framework does not replace analyst judgment. Plugin output is evidence that must be interpreted within context. An investigator should avoid declaring a process malicious solely because one plugin reports an anomaly. Correlation with independent evidence remains essential.

The practical value of Volatility is therefore not simply the ability to print a process list. Its greater value lies in enabling a structured investigation of the runtime state of a computer system.

14. Conclusion

This paper presented a practical tutorial for memory forensics using the Volatility Framework, with a focus on the Volatility 2 repository. The paper explained volatile memory, the plugin-oriented architecture of Volatility, installation considerations, and a workflow for analyzing a Windows memory image.

Several important plugins were discussed, including `pslist`, `psscan`, `pstree`, `dlllist`, `netscan`, `cmdscan`, and `malfind`. The examples demonstrate how process, network, module, and command artifacts can be correlated to support forensic investigation.

Volatility 2 remains useful for learning classic memory-forensics methods, but its archived status and legacy Python requirements should be considered. For contemporary work, Volatility 3 should be evaluated as the current generation of the project. Regardless of framework version, reliable memory forensics depends on sound evidence preservation, reproducible analysis, careful interpretation, and appropriate authorization.

References

[1] Volatility Foundation, *Volatility: An Advanced Memory Forensics Framework*, GitHub repository. https://github.com/volatilityfoundation/volatility

[2] Volatility Foundation, *Volatility Usage*, GitHub Wiki. https://github.com/volatilityfoundation/volatility/wiki/volatility-usage

[3] Volatility Foundation, *Frequently Asked Questions*, GitHub Wiki. https://github.com/volatilityfoundation/volatility/wiki/FAQ

[4] Volatility Foundation, *Volatility Repository*, archived May 16, 2025. https://github.com/volatilityfoundation/volatility

[5] Volatility Foundation, *Volatility Releases*, GitHub. https://github.com/volatilityfoundation/volatility/releases

[6] S. Iqbal and J. M. Hamamreh, *A Comprehensive Tutorial on How to Practically Build and Deploy 5G Networks Using Open-Source Software and General-Purpose, Off-the-Shelf Hardware*, RS-OJICT, 2021. DOI: https://doi.org/10.46470/03d8ffbd.4ccb7950